

Competitive Programming

Section 29: Modular Arithmetic

Competitive Programming Course

2026-07-06

Table of contents

1	Why modular arithmetic starts the math phase	3
2	Remainders and normalization	4
3	Safe multiplication	5
4	Binary exponentiation	6
5	Modular division	7
6	Repeated affine transformations	8
7	Worked example	9
8	Common mistakes	10
9	Local judge workflow	11
10	Submission checklist	12

1 Why modular arithmetic starts the math phase

Competitive programming uses modular arithmetic for two main reasons:

1. Answers are enormous, so the statement asks for the answer modulo M .
2. Division, powers, recurrences, and counting formulas must be computed fast.

Modulo problems punish small implementation mistakes. A correct formula can still fail because of overflow, negative remainders, or illegal division.

2 Remainders and normalization

$x \bmod M$ is the remainder in the range:

```
0, 1, 2, ..., M-1
```

Python already normalizes negative values:

```
(-1) % 7 == 6
```

C++ keeps the sign of the left operand:

```
(-1) % 7 == -1
```

So in C++, normalize after taking %:

```
long long norm(long long x, long long mod) {  
    x %= mod;  
    if (x < 0) x += mod;  
    return x;  
}
```

3 Safe multiplication

If $M \leq 10^9+7$, multiplying two reduced `long long` values is safe because:

$$(10^9+7)^2 < 1.1 \cdot 10^{18} < 9.22 \cdot 10^{18}$$

If a problem allows moduli near 10^{18} , use `__int128` in C++:

```
long long mul_mod(long long a, long long b, long long mod) {  
    return (__int128)a * b % mod;  
}
```

The local **Power Queries** problem allows arbitrary moduli up to 10^9+7 , but the implementation uses `__int128` as a robust habit.

4 Binary exponentiation

Computing a^b by multiplying b times is impossible when b is large.

Binary exponentiation uses the binary representation of the exponent:

```
13 = 8 + 4 + 1 = 1101_2  
 $a^{13} = a^8 * a^4 * a$ 
```

Algorithm:

```
result = 1  
base = a mod M  
while b > 0:  
    if b is odd:  
        result = result * base mod M  
    base = base * base mod M  
    b = floor(b / 2)
```

Each loop halves b , so the complexity is $O(\log b)$.

5 Modular division

Division modulo M does not mean ordinary division followed by $\%$.

To divide by b , we need a number inv such that:

$$b * inv == 1 \pmod{M}$$

Then:

$$a / b == a * inv \pmod{M}$$

An inverse exists exactly when $\gcd(b, M) = 1$. Section 30 will study gcd and the extended Euclidean algorithm. In this section, we use the most common contest case: M is prime and b is not divisible by M .

By Fermat's little theorem:

$$b^{(M-1)} == 1 \pmod{M}$$

Therefore:

$$b^{(M-2)} \text{ is the modular inverse of } b$$

So under prime modulus:

$$\text{inverse}(b) = \text{pow_mod}(b, M-2, M)$$

6 Repeated affine transformations

Suppose:

$$f(x) = a*x + b$$

Apply it repeatedly:

$$\begin{aligned} f(x) &= a*x + b \\ f(f(x)) &= a^2*x + a*b + b \\ f^3(x) &= a^3*x + a^2*b + a*b + b \end{aligned}$$

After k applications:

$$f^k(x) = a^k*x + b*(1 + a + a^2 + \dots + a^{(k-1)})$$

The geometric sum is:

```
if a == 1:
    k
else:
    (a^k - 1) / (a - 1)
```

Under prime modulus, the division by $a - 1$ is multiplication by the modular inverse of $a - 1$.
Be careful: check $a == 1$ **after reducing a modulo M** .

7 Worked example

Let:

$$\begin{aligned}f(x) &= 2x + 3 \\x &= 1 \\k &= 3\end{aligned}$$

Direct simulation:

$$1 \rightarrow 5 \rightarrow 13 \rightarrow 29$$

Formula:

$$\begin{aligned}2^3 * 1 + 3*(1 + 2 + 4) \\&= 8 + 21 \\&= 29\end{aligned}$$

8 Common mistakes

```
Using ordinary division instead of modular inverse.  
Trying to compute  $a^b$  by  $b$  multiplications.  
Forgetting that C++ negative % stays negative.  
Using Fermat inverse when the modulus is not prime.  
Forgetting the  $a == 1$  case in geometric sums.  
Not reducing  $a$ ,  $b$ , and  $x$  before formulas.  
Overflowing int or long long multiplication.
```

9 Local judge workflow

Run:

```
CP_TARGET=solution python3 sections/29_modular_arithmetic/check.py
```

The tests include:

```
modulus 1 for exponentiation  
zero exponent  
small prime moduli for inverse  
a == 1 and a == 0 in affine repetition  
large k values
```

10 Submission checklist

1. Is the modulus prime before using Fermat inverse?
2. Are all values normalized into $[0, M-1]$?
3. Does exponentiation run in $O(\log \text{exponent})$?
4. Does C++ multiplication fit, or do you need `__int128`?
5. Did you handle special cases: $M=1$, $\text{exponent}=0$, $a=1$?